

How Transformers work?

Goal to understand :

Self-attention mechanism

Vector embedding

Feed forward Neural Network

A transformer is a type of artificial intelligence model that learns to understand and generate human-like text by analyzing patterns in large amounts of text data.

Transformers have revolutionized the field of deep learning, especially in Natural Language Processing (NLP), by enabling models like BERT, GPT, and T5 to process language with remarkable accuracy.

At the heart of a Transformer are three key concepts: Vector Embedding, Self-Attention Mechanism, and Feed-Forward Neural Networks.

1. Vector Embeddings-Turning Words into Numbers

Computers cannot directly understand words; they work with numbers.

Vector embedding is the process of converting each word (or token) into a numerical vector that captures its meaning.

- Each word, sentence, or image is turned into a vector (a list of numbers).
- Each word is mapped to a high-dimensional vector (e.g., 512 or 1024 dimensions).
- Words with similar meanings have vectors that are close together in this space.
- Example: The vectors for “king” and “queen” will be closer than “king” and “banana”.
- Similar concepts are placed close together: “cat” and “dog” are neighbors, while “cat” and “car” are far apart.
- This allows the model to “understand” relationships between words beyond dictionary definitions.
- Embeddings allow the model to understand semantic relationships.
- They serve as the **input layer** for the Transformer.

Analogy: Imagine a huge 3D map where every word is a dot. Words with similar meaning cluster together like cities in the same region.

2. Self-Attention-Finding Contextual Relationships

The **self-attention mechanism** is the core innovation of Transformers.

It allows the model to look at **all words in a sentence at once** and decide how much each word should influence the others.

Self-attention is the **spotlight mechanism**.

For each word, the model asks: *“Which other words in this sentence are important for understanding me?”*

How it works:

Three vectors are created for each word:

- **Query (Q)** – What am I looking for?
- **Key (K)** – What do I contain?
- **Value (V)** – What information do I carry?
- **Attention scores** are calculated by comparing Q and K (via dot product).
- Scores are normalized using **softmax** to get attention weights.
- Each word’s output is a **weighted sum of the Value vectors** of all words.

Example:

- In the sentence *“The cat sat on the mat”*, when processing “cat,” the model might give high attention to “sat” and “mat” but low attention to “the.”

Why it matters:

- Captures **long-range dependencies** (e.g., linking a pronoun to a noun far away in the text).
- Works in parallel, unlike RNNs which process sequentially.
- Each word looks at all other words in the sentence and decides which ones matter most.
- It uses **queries, keys, and values** to calculate attention scores.
- Example: In *“The animal didn’t cross the road because it was too tired”*, the word “it” pays more attention to “animal” than “road”.

Analogy: Imagine a group conversation. Each person listens to everyone else but focuses more on the voices that are most relevant to what they’re saying.

3. Feed-Forward Neural Networks-Refining the Representation

After self-attention, each word's vector is passed through a position-wise feed-forward neural network.

After attention decides what's important, the information flows through a **feed-forward neural network**.

Structure:

- Two linear (dense) layers with a non-linear activation (often ReLU or GELU) in between.
- Applied **independently** to each position (word) but with the same weights.

Purpose:

- Adds non-linear transformations to capture more complex patterns.
- Helps the model combine and refine the contextual information learned from self-attention.
- This is like a **factory conveyor belt**: data enters, gets transformed layer by layer, and exits as a refined signal.
- Each layer applies mathematical functions (like ReLU or sigmoid) to capture complex patterns.
- It ensures the model doesn't just "copy" attention scores but learns deeper representations.

Analogy: Think of it as a chef taking raw ingredients (attention outputs) and cooking them into a finished dish (final representation).

The Transformer Block:

A Transformer encoder block typically follows this sequence:

1. Input Embedding → Convert tokens to vectors.
2. Positional Encoding → Add position information (since self-attention is order-agnostic).
3. Multi-Head Self-Attention → Multiple attention layers run in parallel to capture different types of relationships.
4. Add & Norm → Residual connection + Layer normalization.

5. Feed-Forward Neural Network → Non-linear transformation of each position.
6. Add & Norm → Another residual connection + normalization.

Multiple such blocks are stacked to form the full Transformer.

Why This Matters

- **Vector Embedding** gives meaning to words.
- **Self-Attention** lets the model understand relationships between words anywhere in the text.
- **Feed-Forward Networks** refine and transform these relationships into richer representations.

This architecture has made Transformers the **foundation of modern AI models** in NLP, vision, and even multimodal tasks.

Quick Analogy

Think of reading a paragraph:

- **Embedding:** You know the meaning of each word